



Proof of Concept

Administración de un Cluster Kubernetes con Ansible

72.20 Redes de Información

BENGOLEA, IÑAKI (63515), IBENGOLEA@ITBA.EDU.AR
BRAUN, SANTOS (62090), SBRAUN@ITBA.EDU.AR
LOPEZ MENARDI, FELIX (62707), FLOPEZMENARDI@ITBA.EDU.AR

23 de Septiembre, 2025

1 ¿Qué es Ansible y por qué lo elegimos?

Ansible es una herramienta de automatización de infraestructura que permite orquestar la configuración y gestión de múltiples servidores a través de playbooks, los cuales son simples archivos YAML que describen de forma declarativa qué se quiere lograr en cada máquina. Cada playbook está compuesto por plays (qué hosts van a ser configurados) y dentro de ellos por tasks (acciones concretas como instalar paquetes, copiar archivos, configurar servicios, etc.). Estas tasks a su vez se apoyan en módulos predefinidos de Ansible, que son piezas reutilizables de código que ejecutan la acción en el host remoto.

La principal alternativa que podríamos haber considerado sería manejar el cluster con el CLI de Kubernetes (kubectl), pero esto implicaría que la provisión inicial de nodos, instalación de dependencias y configuración debería hacerse manualmente en cada máquina. Tampoco se contaría con un mecanismo centralizado para repetir o versionar la instalación.

Al usar Ansible obtenemos:

- **Automatización completa** de la creación y configuración de nodos.
- **Idempotencia:** si ejecutamos el playbook varias veces, siempre nos deja en el mismo estado deseado.
- **Escalabilidad:** con solo agregar nuevas IPs al inventario, podemos expandir el cluster.
- **Menos fricción:** evitamos pasos manuales y aseguramos consistencia entre entornos.

Por estas razones, Ansible fue la mejor elección para este proyecto.

2 Arquitectura General

La arquitectura de nuestro sistema consta de:

1. **Control Node (Ansible):** es nuestra propia computadora local, desde donde ejecutamos los playbooks.
2. **Virtual Machines (VMs):** cada una representa un nodo dentro del cluster Kubernetes.
 - Una VM cumple el rol de **Control Plane Node**, es decir que gestiona el API Server, el scheduler y el etcd del cluster.
 - Las demás VMs cumplen el rol de **Worker Nodes** donde efectivamente corren los **Pods** (unidades mínimas de despliegue de Kubernetes).
 - (a) Dentro de cada Pod corre uno o varios contenedores, normalmente 1 contenedor = 1 microservicio. Los contenedores son ejecutados por un container runtime (containerd).

- (b) Dentro de cada nodo, un agente (kubelet) se comunica con el control plane para recibir instrucciones y gestionar el ciclo de vida de los Pods.
- (c) Es posible que un mismo Pod tenga contenedores adicionales llamados sidecars, que realizan tareas auxiliares como logging o monitoreo, acompañando al contenedor principal del microservicio.

Cada VM tendrá una dirección IP fija incluida en nuestro inventario de Ansible. A través de este inventario, Ansible se conecta vía SSH a cada VM y ejecuta las configuraciones necesarias.

Decidimos usar VMs locales (Multipass) en lugar de hostear en un cloud provider (ej. AWS) por cuestiones de simplicidad, control y reproducibilidad. Integrar Ansible directamente con AWS requiere configurar credenciales, redes y seguridad (típicamente con Terraform, que además es el tema de otro grupo). Con VMs evitamos costos, dependencias externas y garantizamos que cualquiera pueda replicar nuestro PoC.

Dejamos planteada una migración futura a cloud como ampliación natural: Terraform para la provisión de instancias en AWS y Ansible para el bootstrap y la administración del cluster.

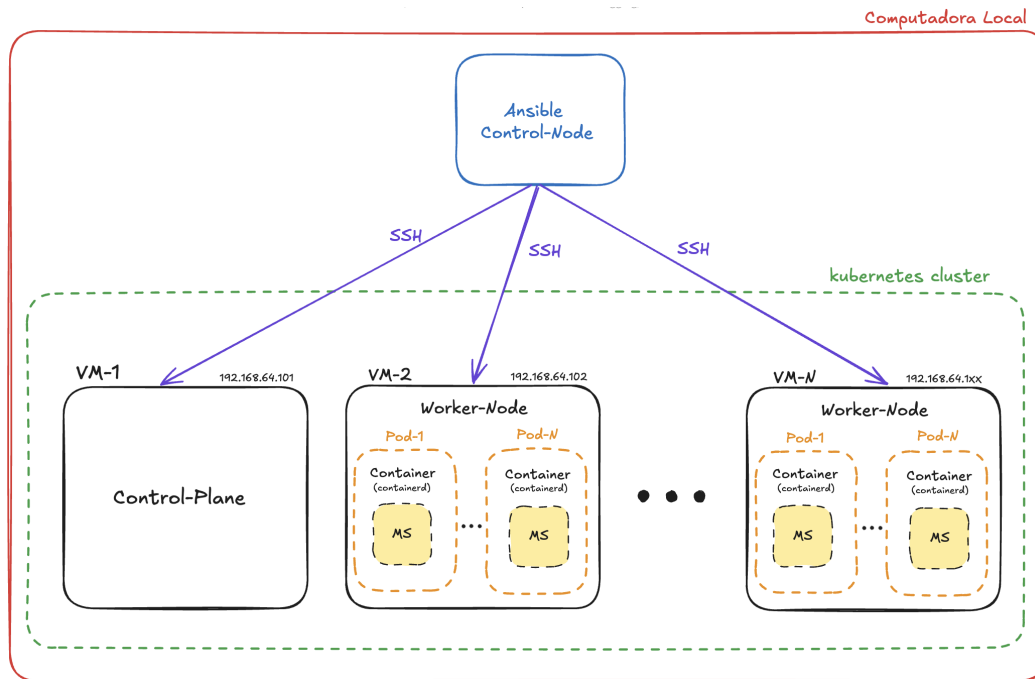


Figure 1: Diagrama detallado de la arquitectura

3 Elección de Multipass

Todos los integrantes del grupo trabajamos en computadoras Mac con chip Apple Silicon.

Consideramos distintas alternativas:

- **VirtualBox** acaba de añadir soporte para esta arquitectura hace pocos meses, lo que lo hace más riesgoso, y además sería necesario combinarlo con Vagrant ya que Ansible no cuenta con un módulo nativo para VirtualBox.
- **VMware Fusion**, por su parte, cuenta con módulos dedicados en Ansible, pero están pensados principalmente para entornos vSphere/ESXi y no se ajustan a nuestro objetivo.

Por este motivo optamos por usar **Multipass**, una herramienta ligera y estable que permite crear VMs de Ubuntu de manera rápida y sencilla. Con Multipass podemos levantar nuevas VMs con un solo comando (`multipass launch`), integrarlas fácilmente al inventario de Ansible, e incluso ejecutar comandos de Multipass directamente desde las tasks de un playbook para su provisión automática.

4 Comunicación en el Cluster

- **Entre VMs:** todas las VMs estarán en la misma subred (192.168.64.0/24) lo cual asegura conectividad directa entre nodos.
- **Entre Control Plane y Workers:** el Control Plane coordina el estado del cluster, mientras que los Workers ejecutan los Pods y reportan su estado al Control Plane.
- **Entre Pods:** Kubernetes usará el CNI (Container Network Interface) **Flannel** para crear una red virtual que permite que los Pods se comuniquen entre sí, incluso si se encuentran en distintos nodos físicos.

De esta forma, logramos que tanto la capa de infraestructura (VMs) como la capa de orquestación (Kubernetes) estén correctamente conectadas y funcionando de manera integrada.

5 Elección de la tecnología de instalación

Se analizaron las siguientes alternativas:

- **kubeadm:** herramienta oficial de Kubernetes que permite inicializar un cluster "vanilla" paso a paso. Requiere instalar dependencias, configurara redes, CRI, etc., VM por VM. Es más completo pero más complejo y necesita mayor atención al detalle.

- **Kubernetes completo (k8s):** implica instalar manualmente todos los componentes del cluster (API Server, Controller Manager, Scheduler, etcd, kubelet, kube-proxy, container runtime), más todas las configuraciones de red y almacenamiento. Muy robusto pero demasiado pesado para un entorno pequeño de PoC local.
- **k3s:**
 - Versión ligera de Kubernetes que incluye todos los componentes esenciales del cluster en un solo paquete y con menos dependencias.
 - Ideal para entornos pequeños como nuestro PoC con pocas VMs, ya que reduce la complejidad de instalación y configuración, simplifica actualizaciones y ocupa menos recursos.
 - Sigue siendo Kubernetes compatible, por lo que todos los conceptos de pods, deployments y servicios funcionan igual que en un cluster completo.

Optamos por k3s por su simplicidad, rapidez de despliegue y compatibilidad con Kubernetes estándar, al mismo tiempo evitando la sobrecarga de configurar todo manualmente con kubeadm o k8s.

6 Playbooks del Proyecto

En nuestro proyecto cada playbook de Ansible representa una capacidad concreta de administración del cluster. Decidimos estructurarlos de forma modular para que cada archivo YAML sea responsable de un aspecto puntual de la infraestructura. Esto no solo mejora la legibilidad y el mantenimiento, sino que también facilita la reutilización en futuros proyectos.

Los playbooks definidos son:

- **provision.yml**
 - Instala dependencias básicas en las VMs (paquetes de red y el binario de k3s).
 - Asegura que todas las máquinas tengan la configuración de sistema operativo y usuarios necesaria para unirse al cluster.
- **control-plane.yml**
 - Inicializa el nodo maestro con k3s.
 - Configura el API Server, guarda el join token que usarán otros nodos para conectarse y también guarda el archivo kubeconfig para permitir la administración remota del cluster.
- **join-workers.yml**
 - Toma el token de join generado por el control plane.
 - Conecta automáticamente nuevos nodos al cluster, dejándolos listos para ejecutar Pods.

- **scale.yml**
 - Automatiza el agregado o eliminación de nodos de forma declarativa.
 - Permite al usuario ajustar manualmente la capacidad del clúster ejecutando un playbook que crea o destruye nodos (VMs).
- **status.yml**
 - Consulta el estado general del cluster (nodos disponibles, Pods activos, servicios desplegados).
 - Sirve como “salud del sistema” accesible desde el control node.

Con esta estructura buscamos que el cliente pueda administrar el ciclo de vida completo del cluster de manera declarativa, ejecutando un playbook distinto según la operación deseada.

Cabe destacar que no se incluye un playbook específico de 'networking' para el despliegue del CNI (Flannel), dado que es una acción realizada directamente por k3s cuando se inicializan los nodos.

De esta forma, los playbooks representan todo lo que ofrecemos como equipo: una solución integral de administración de cluster Kubernetes, reproducible, escalable y automatizada.

7 Relación con la aplicación The Store

La cátedra provee como aplicación de referencia The Store, compuesta por múltiples microservicios que simulan un sitio de comercio electrónico. Esta aplicación nos sirve como caso de uso realista para validar clusters de Kubernetes.

Nuestro proyecto no incluye el despliegue de The Store, ya que el foco está en la automatización y administración de la infraestructura. Sin embargo, el cluster que provisionamos queda preparado para ejecutar aplicaciones de ese estilo.

Nuestro plan es demostrar:

- Cómo el cluster puede escalar horizontalmente (ej. agregando un worker node con el playbook scale.yml).
- Cómo es posible verificar el estado de los nodos y pods mediante el playbook status.yml.
- Que la comunicación entre nodos y pods funciona correctamente gracias a la red provista por el CNI.

De esta forma, si un cliente quisiera ejecutar The Store (u otra aplicación multi-servicio), podría hacerlo directamente sobre nuestra infraestructura, validando distribución de pods, comunicación interna y balanceo de carga. Así, el cluster no es un entorno vacío, sino una plataforma lista para hospedar aplicaciones reales.